

SSH Brute Force Attack Detection Using Hydra and Wazuh

By Varun Wilfred

1. Introduction

A **brute force attack** is a technique used by attackers to gain unauthorized access by repeatedly attempting different username and password combinations until the correct credentials are discovered. Although simple in nature, brute force attacks remain one of the most common methods used to compromise remote services such as **SSH, FTP, RDP**, and web applications.

Secure Shell (SSH) is widely used for secure remote administration of systems. If weak passwords are used or login attempts are not restricted, SSH services become attractive targets for brute force attacks.

Disclaimer: The following exercise was performed entirely within a personal virtual laboratory for educational purposes. All attacks were conducted against systems owned and configured by the author. Unauthorized attacks against systems without permission are illegal and unethical.

2. Types of Brute Force Attacks

The most common brute force attack techniques include:

- **Simple Brute Force Attack** – Directly attempting all possible combinations of passwords
- **Dictionary Attack** – Uses a predefined list of common passwords instead of trying every possible combination.
- **Hybrid Attack** – Combines dictionary words with numbers, symbols, or character substitutions.
- **Reverse Brute Force Attack** – Uses one common password against multiple usernames.
- **Credential Stuffing** – Uses username-password combinations leaked from previous data breaches.

3. Popular Brute Force Tools

Several security tools can perform password auditing and brute force testing.

- * Hydra
- * John the Ripper
- * Hashcat
- * Medusa
- * Ncrack
- * Aircrack-ng

For this project, **Hydra** was selected because of its speed, protocol support, and suitability for SSH authentication testing.

4. Hydra

Hydra is an open-source network login cracker capable of performing dictionary-based authentication attacks against numerous network services.

Hydra supports protocols including:

- * SSH
- * FTP
- * HTTP
- * HTTPS
- * SMB
- * Telnet
- * RDP
- * VNC
- * POP3
- * IMAP
- * SMTP

Its flexibility makes it one of the most widely used password auditing tools in penetration testing environments.

4.1 Hydra Command Structure

```
hydra -l <username> -P <password_file> ssh://<target_ip>
```

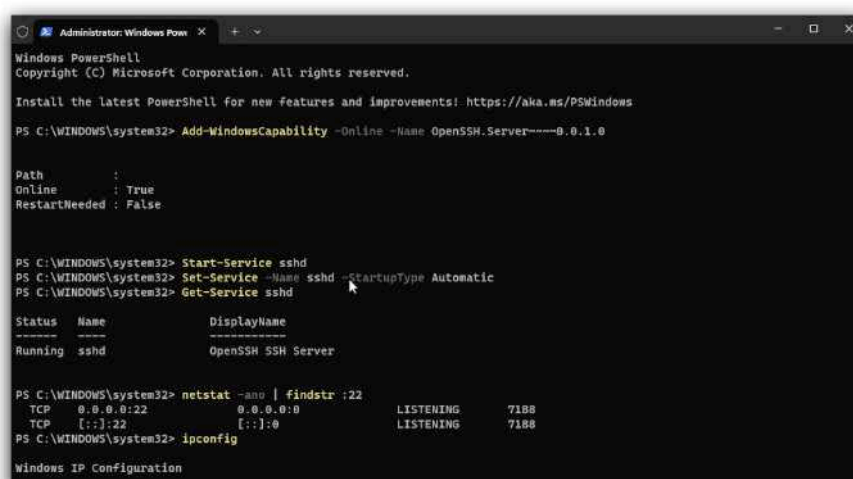
Where:

- **-l** : specifies a single username.
- **-P** : specifies the password wordlist.
- **ssh://** : identifies the target service.
- **target_ip** : is the IP address of the Windows endpoint.

5. SSH Brute Force Attack

An SSH brute force attack repeatedly attempts different passwords against the SSH service running on the target system until authentication succeeds or the password list is exhausted.

In this project, the Windows endpoint was configured with the OpenSSH Server feature, allowing remote SSH connections from the Kali Linux attacker machine.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0

Path      :
Online    : True
RestartNeeded : False

PS C:\WINDOWS\system32> Start-Service sshd
PS C:\WINDOWS\system32> Set-Service -Name sshd -StartupType Automatic
PS C:\WINDOWS\system32> Get-Service sshd

Status Name      DisplayName
-----
Running sshd      OpenSSH SSH Server

PS C:\WINDOWS\system32> netstat -ano | findstr :22
TCP    0.0.0.0:22      0.0.0.0:0      LISTENING  7188
TCP    [::]:22       [::]:0        LISTENING  7188
PS C:\WINDOWS\system32> ipconfig

Windows IP Configuration
```

6. System Requirements

Host Machine

- * macOS (MacBook Air M2)

Virtualization Platform

- * UTM

Attacker Machine

- * Kali Linux

Target Machine

- * Windows 11
- * OpenSSH Server Enabled
- * Wazuh Agent Installed

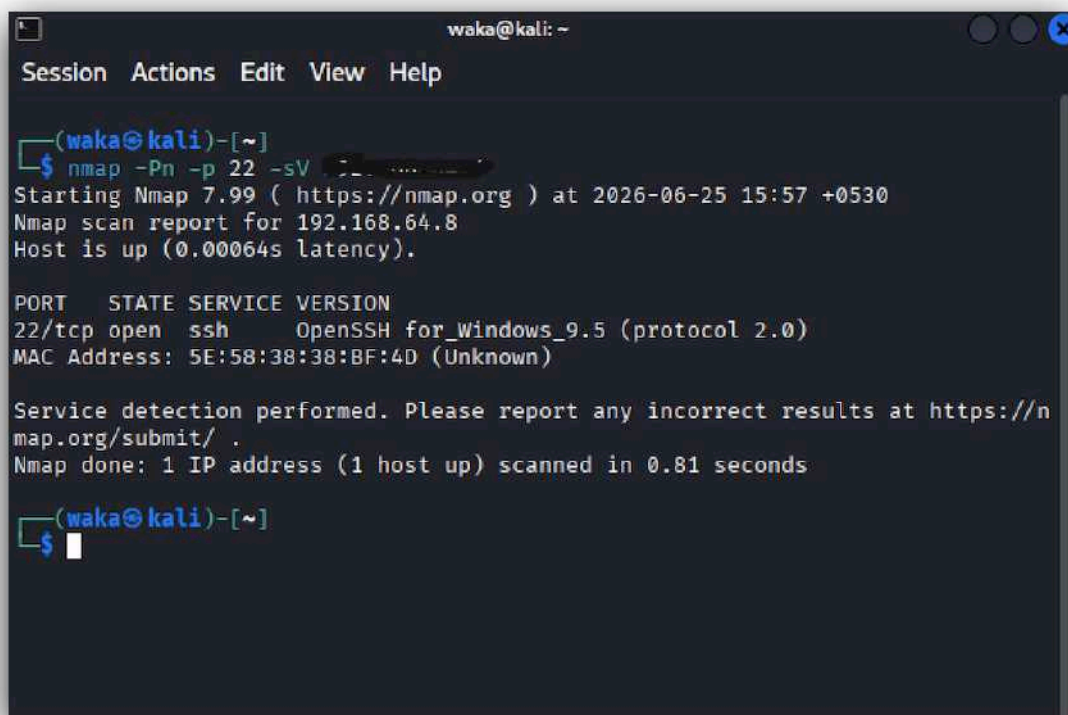
7. Simulating an SSH Brute Force Attack

7.1 Step 1: Verify SSH Availability

Before launching the attack, the SSH service running on the Windows endpoint was verified using Nmap.

The scan confirmed that:

- * Port 22 was open.
- * OpenSSH for Windows was running.
- * The Windows machine was reachable from Kali Linux.



```
waka@kali: ~  
Session Actions Edit View Help  
  
(waka@kali)-[~]  
$ nmap -Pn -p 22 -sV 192.168.64.8  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-25 15:57 +0530  
Nmap scan report for 192.168.64.8  
Host is up (0.00064s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH for_Windows_9.5 (protocol 2.0)  
MAC Address: 5E:58:38:38:BF:4D (Unknown)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 0.81 seconds  
  
(waka@kali)-[~]  
$
```

Figure : Nmap scan confirming the SSH service is available.

7.2 Step 2: Create the Password List

A password list named **passwords.txt** was created on Kali Linux.

Example:

```
```text
123456
password
admin
qwerty
letmein
```
```

The correct password was intentionally omitted to generate failed authentication events.

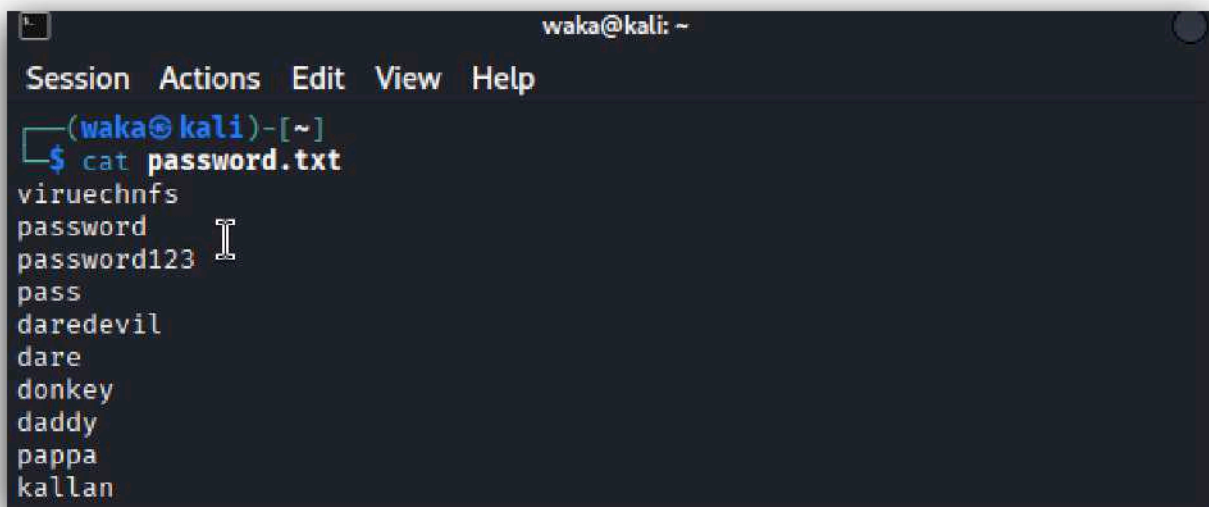
A terminal window titled 'waka@kali: ~' showing the command 'cat password.txt' and its output. The output lists several passwords: viruechnfs, password, password123, pass, daredevil, dare, donkey, daddy, pappa, and kallan. A cursor is visible after 'password123'.

Figure : Password list used during the brute force simulation.

7.3 Step 3: Execute Hydra

The Windows username used in the lab was **deep**.

Hydra was executed using:

```
hydra -l deep -P passwords.txt ssh://< target_ip >
```

Hydra attempted multiple authentication requests using the supplied password list.

Since all passwords were incorrect, every login attempt failed.

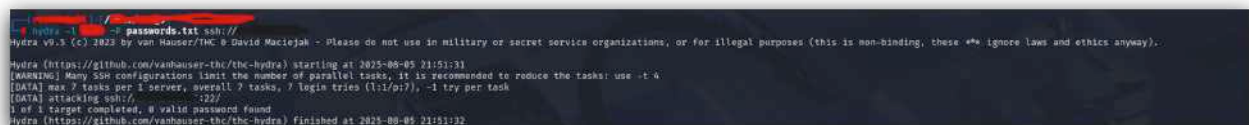
A terminal window showing the execution of Hydra. The output includes the Hydra version (V9.3), a warning about legal use, the start time (2025-08-05 23:51:31), and the command being executed: 'hydra -l deep -P passwords.txt ssh://< target_ip >'. It also shows the number of tasks (7) and login tries (1) per task, and the fact that no valid passwords were found after 7 attempts.

Figure : Hydra performing multiple SSH authentication attempts.

7.4 Step 4: Observe Authentication Failures

Each failed authentication attempt generated Windows Security events.

These events were collected by the Wazuh Agent and forwarded to the Wazuh Manager for analysis.

8. Detection in Wazuh

After executing Hydra, the Wazuh Dashboard was opened.

Navigate to:

Threat Hunting

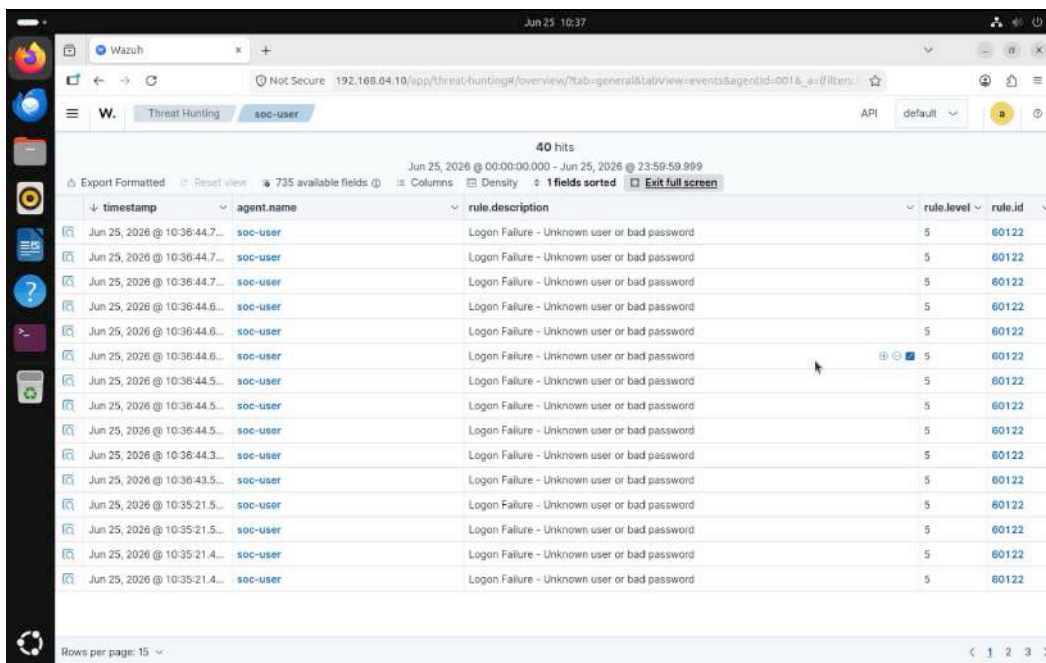
The dashboard displayed multiple authentication failure events generated during the attack.

Observed alerts included:

- * Failed SSH logon attempts
- * Authentication failures
- * Unknown username or bad password
- * Windows Security Event ID 4625

Each event contained:

- * Timestamp
- * Source Agent (soc-user)
- * Event ID
- * Rule Description
- * Severity Level
- * Source IP Address
- * Destination Service (OpenSSH)



The screenshot shows the Wazuh Threat Hunting dashboard with a table of 40 hits. The table columns are timestamp, agent.name, rule.description, rule.level, and rule.id. All entries show 'Logon Failure - Unknown user or bad password' with a severity level of 5 and rule ID 60122. The agent name for all entries is 'soc-user'.

| timestamp | agent.name | rule.description | rule.level | rule.id |
|------------------------------|------------|--|------------|---------|
| Jun 25, 2026 @ 10:36:44.7... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.7... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.7... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.6... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.6... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.6... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:44.3... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:36:43.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:35:21.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:35:21.5... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:35:21.4... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |
| Jun 25, 2026 @ 10:35:21.4... | soc-user | Logon Failure - Unknown user or bad password | 5 | 60122 |

Figure : Wazuh detecting multiple failed SSH authentication attempts.

9. Windows Security Event Analysis

Event Details

The primary Windows event generated during the attack was **Event ID 4625 (An account failed to log on).**

Important fields observed during analysis included:

| Field name | Field value | Meaning |
|-------------------------------------|--------------------------------------|---|
| data.win.eventdata.failureReason | %%2313 | Unknown user name or bad password. |
| data.win.eventdata.keyLength | 0 | No encryption key was used. |
| data.win.eventdata.logonProcessName | Advapi | Typical for services, not interactive logins. |
| data.win.eventdata.logonType | 8 | Logon type 8 = NetworkCleartext. Credentials were sent in clear text over the network (common with SSHD). |
| data.win.eventdata.processId | 0x65d8 | The process ID (0x65d8) of the process that attempted the logon. |
| data.win.eventdata.processName | C:\Windows\System32\OpenSSH\sshd.exe | Confirms someone tried to connect to this machine over SSH. |
| data.win.eventdata.status | 0xc000006d | Status = Logon failure. |
| data.win.eventdata.subStatus | 0xc0000064 | User logon with unknown user name OR bad password. |
| data.win.system.channel | Security | The event comes from the Windows Security log. |
| data.win.system.eventID | 4625 | Event ID 4625 = An account failed to log on. |
| data.win.system.eventRecordID | 919137 | A unique sequential ID for this event in the event log. |
| data.win.system.systemTime | 2025-08-05T20:51:34.3997254Z | Event timestamp (UTC). |
| decoder.name | windows_eventchannel | Wazuh parsed this as a Windows EventChannel log. |
| input.type | log | Confirms this came from a log source (Windows logs). |
| rule.firedtimes | 14 | At least 14 failed logins of this type have occurred. |

Table : Windows Event 4625 – Failed Logon Attempt

Summary

- On **25-06-2026** at **10:36:44 IST**, the Windows machine MyPc received an SSH login attempt via sshd.exe.
- Authentication failed with error Unknown username or bad password (0xc0000064).
- This was logged as Event ID 4625 (failed logon).
- Wazuh classified it under failed authentication events, severity level 5, and mapped it to multiple compliance frameworks.
- The rule has already fired 14 times, so this is not an isolated event.

10. Defensive Countermeasures

The following security measures help reduce the risk of SSH brute force attacks:

- * Use strong and unique passwords.
- * Enable Multi-Factor Authentication (MFA).
- * Disable password authentication and use SSH keys where possible.
- * Restrict SSH access using firewall rules.
- * Change the default SSH configuration where appropriate.
- * Implement account lockout policies.
- * Monitor authentication logs regularly.
- * Configure automated alerting through SIEM platforms such as Wazuh.

11. Conclusion

This report demonstrated how an SSH brute force attack can be simulated using Hydra against a Windows 11 endpoint running the OpenSSH Server service. The attack was performed from a Kali Linux virtual machine within a controlled laboratory environment.

During the simulation, Hydra generated multiple failed authentication attempts, which were recorded by the Windows Security log. The Wazuh Agent collected these events and forwarded them to the Wazuh Manager, where they were successfully detected and displayed in the Threat Hunting dashboard.

This exercise shows the importance of monitoring and detection tools like Wazuh in strengthening system defenses. While brute force attacks are relatively simple, they can still be effective if strong passwords and proper security measures are not in place. In future work, deeper analysis of Wazuh logs and alerting mechanisms will provide further insights into identifying, investigating, and mitigating brute force threats.